

```

"""8-puzzle solved with depth-limited DFS and iterative deepening (IDS).

Boards are tuples of 9 numbers read row by row; 0 is the blank.
Moves are the blank sliding Up, Down, Left, Right, in that order.
"""

START = (2, 8, 3,
         1, 6, 4,
         7, 0, 5)
GOAL = (1, 2, 3,
        8, 0, 4,
        7, 6, 5)

MOVES = [("U", -3), ("D", 3), ("L", -1), ("R", 1)]

def neighbors(board):
    """Yield (move, new_board) for every legal blank move."""
    z = board.index(0)
    for name, d in MOVES:
        if name == "U" and z < 3:
            continue
        if name == "D" and z > 5:
            continue
        if name == "L" and z % 3 == 0:
            continue
        if name == "R" and z % 3 == 2:
            continue
        b = list(board)
        b[z], b[z + d] = b[z + d], b[z]
        yield name, tuple(b)

def depth_limited_dfs(board, goal, limit, path, on_path, stats):
    """Recursive DFS. Returns True when the goal is found; path holds the moves.

    on_path stops the search from looping back to a state on the current path.
    """
    stats["expanded"] += 1
    if board == goal:
        return True
    if len(path) >= limit:
        return False
    for move, nxt in neighbors(board):
        if nxt in on_path:
            continue
        path.append(move)
        on_path.add(nxt)
        if depth_limited_dfs(nxt, goal, limit, path, on_path, stats):
            return True
        path.pop()
        on_path.remove(nxt)
    return False

def dfs(start, goal, limit):
    """DFS with a fixed depth limit."""
    stats = {"expanded": 0}
    path = []
    found = depth_limited_dfs(start, goal, limit, path, {start}, stats)
    return (path if found else None), stats["expanded"]

def ids(start, goal, max_depth=31):
    """Iterative deepening: run DFS with limit 0, 1, 2, ... until it succeeds."""
    stats = {"expanded": 0}
    for limit in range(max_depth + 1):
        path = []
        if depth_limited_dfs(start, goal, limit, path, {start}, stats):
            return path, stats["expanded"], limit
    return None, stats["expanded"], max_depth

def solvable(start, goal):
    """A pair is solvable when both boards have the same inversion parity."""
    def inversions(b):
        t = [x for x in b if x]
        return sum(t[i] > t[j] for i in range(8) for j in range(i + 1, 8)) % 2
    return inversions(start) == inversions(goal)

```

```

def show(board):
    for r in range(0, 9, 3):
        print(" ".join(str(x) if x else "_" for x in board[r:r + 3]))

if __name__ == "__main__":
    if not solvable(START, GOAL):
        raise SystemExit("These boards are unsolvable.")

    print("Start:")
    show(START)
    print("Goal:")
    show(GOAL)

    path, expanded = dfs(START, GOAL, limit=8)
    print(f"\nDFS (limit 8): moves={''.join(path) if path else None}, "
          f"cost={len(path) if path else '-'}, expanded={expanded}")

    path, expanded, depth = ids(START, GOAL)
    print(f"IDS: moves={''.join(path) if path else None}, "
          f"cost={len(path) if path else '-'}, expanded={expanded}, final limit={depth}")

```

Start:

2 8 3

1 6 4

7 _ 5

Goal:

1 2 3

8 _ 4

7 6 5

DFS (limit 8): moves=UULDR, cost=5, expanded=14

IDS: moves=UULDR, cost=5, expanded=73, final limit=5